

1° CUATRIMESTRE

📝📺🛠️ ACTIVIDAD 5: MODELADO DE DATOS CON ESTRUCTURAS (STRUCT) Y PUNTEROS - LPR 5° 3° A-B (Presencial/Virtual Asincrónico - Actividades de Contingencia y Continuidad Pedagógica) 2026

🎯 OBJETIVOS DE LA ACTIVIDAD

Modelar entidades complejas del mundo real (por ejemplo sensores, módulos de hardware, usuarios, etc) utilizando estructuras de datos (struct) en C++, controlando el ingreso y la modificación de sus miembros por referencia mediante punteros y el operador flecha (->).

💡 Conexión con el Proyecto Integrador Anual:

Esta actividad sienta algunas bases para estructurar las variables principales de su prototipo de proyecto para la Expotécnica en caso de usar ese lenguaje de programación. Todo lo aprendido aquí debe documentarse en un archivo sugerido en la ruta **docs/manuales/manual_programador_v1.0.0.pdf** o **md** de su repositorio grupal del proyecto integrador anual o uno nuevo en caso de no utilizar esta actividad para su P.I.A..

1. **Planilla de Seguimiento LPR 5° 3° (Proyectos A-B):** Se les recuerda que el líder de cada equipo o al que este designe debe actualizar la fila correspondiente a su grupo, detallando el estado del *Primer/Segundo Avance de Proyecto Integrador Anual, los progresos* y vinculando los nuevos componentes lógicos desarrollados.
2. **Portafolio en GitHub:** Cada ejercicio de esta guía debe estar respaldado por un *commit* descriptivo en su repositorio. No se aceptarán entregas finales que no muestren un historial de desarrollo ramificado (***main*** **y** ***development***) para su P.I.A o la actividad 5 aparte.

📕 GLOSARIO TÉCNICO (para aprender)

- **Struct (Estructura):** Colección de variables de distintos tipos agrupadas bajo una misma entidad lógica en bloques contiguos de memoria RAM.
- **Operador Flecha (->):** Sintaxis exclusiva de C++ para acceder a los campos de una estructura a través de un puntero desreferenciado.
- **Pasaje por Referencia:** Enviar la dirección de memoria (&) de un objeto a una función para modificar el original sin crear duplicados pesados en la RAM.
- **Buffer de Entrada:** Espacio temporal en memoria donde el sistema guarda los caracteres ingresados por teclado.
- **cin.ignore():** Función que limpia el Enter (\n) remanente en el buffer para evitar que la lectura de cadenas de texto (cin.getline) se omita automáticamente.

🗂️ ESTRUCTURA DEL REPOSITORIO EN GITHUB (nombre sugerido sistemaproyecto o el nombre de su P.I.A.)

sistemaproyecto/
```
├── .gitignore                  <-- Reglas de exclusión (.exe, .vscode/)
├── LICENSE                     <-- Licencia MIT escolar
├── README.md                   <-- Portada con integrantes y descripción del modelo
├── docs/                       <-- Carpeta de documentación formal
│   └── InformeEEST1_LPR2026_ACT05_G03_v1.0.0.pdf <-- ENTREGABLE EN PDF (APA v7)
│   └── manuales/
│       └── manual_programador_v1.0.0.pdf <-- Explicación del struct y operador ->
│       └── manual_programador_v1.0.0.md <-- Explicación del struct y operador ->
├── src/                        <-- Código fuente C++
│   └── main.cpp                <-- Programa principal modular
└── capturas/                   <-- Evidencias de ejecución (.png)
    └── ejecucion_struct.png
```

💻 CÓDIGO FUENTE EN C++ (src/main.cpp)

```
/*
  ================================================================
  Materia: Laboratorio de Programación (LPR) — 5°
  Archivo: src/main.cpp
  Actividad 5: Modelado de datos con Structs y Punteros
  ================================================================
*/

#include <iostream>
#include <cstring>

using namespace std;

// 1. DEFINICIÓN DE LA ESTRUCTURA (Modelo sugerido para el Proyecto Integrador Anual)
struct EntidadProyecto {
    int id;
    char nombre[50];
    float metrica; // Representa lecturas de sensores, consumo, avance, etc.
};

// Prototipo de función con pasaje por dirección mediante puntero
void cargarDatos(EntidadProyecto* ptr);

int main() {
    // Inicialización de seguridad en la memoria Stack
    EntidadProyecto miEntidad = {0, "Vacio - [Su Nombre y Apellido]", 0.0f};

    cout << "======================================================" << endl;
    cout << "  MODELADO STRUCT - ESTUDIANTE: [Su Nombre y Apellido] " << endl;
    cout << "======================================================" << endl;

    // Invocación enviando la dirección física de memoria con '&'
    cargarDatos(&miEntidad);

    cout << "\n=== DATOS VERIFICADOS EN LA MEMORIA RAM ===" << endl;
    cout << "ID Registrado:     " << miEntidad.id << endl;
    cout << "Nombre Registrado: " << miEntidad.nombre << endl;
    cout << "Metrica Guardada:  " << miEntidad.metrica << endl;
    cout << "Direccion RAM Hexadecimal: " << &miEntidad << endl;
    cout << "======================================================" << endl;

    return 0; // Código 0: Finalización exitosa
}

void cargarDatos(EntidadProyecto* ptr) {
    cout << "\n--- INGRESO DE DATOS MEDIANTE OPERADOR FLECHA ---" << endl;

    cout << "=> Ingrese el ID de la entidad (entero): ";
    cin >> ptr->id;

    // Limpieza del buffer obligatoria antes de cin.getline
```

```
    cin.ignore();

    cout << "=> Ingrese el Nombre o Descripcion: ";
    cin.getline(ptr->nombre, 50);

    cout << "=> Ingrese la Metrica de Operacion (decimal/float): ";
    cin >> ptr->metrica;
}
```

🏃 COMANDOS DE POWERSHELL DESDE VSCODE PARA COMPILAR Y EJECUTAR

```
# 1. Compilar el programa generando el ejecutable en src/
g++ src/main.cpp -o src/sistemaproyecto.exe

# 2. Ejecutar la aplicación en Windows desde VSCODE
.\src\sistemaproyecto.exe
```

📋 CUESTIONARIO TÉCNICO DE CONTROL

1. **Heterogeneidad:** ¿Qué ventaja ofrece agrupar los datos de un sensor o dispositivo en un struct en lugar de declarar muchas variables sueltas?
2. **Operador Flecha:** ¿Por qué el compilador de C++ no permite usar la notación de punto (ptr.id) cuando la variable es de tipo puntero?
3. **Eficiencia en RAM:** ¿Por qué el pasaje por referencia mediante punteros optimiza la memoria procesada frente al pasaje por copia?
4. **Buffer de Entrada:** ¿Qué comportamiento erróneo ocurre en la terminal si se omite la instrucción cin.ignore() antes de un getline?
5. **Aplicación al Proyecto Integrador:** ¿Qué entidad o componente hardware de su Proyecto Integrador Anual representaron con esta estructura y qué variables componen sus miembros?

🏆 PAUTA DE ENTREGA EN MOODLE

⚠️ Modalidad y Entorno Híbrido:

- **Formato de Entrega del informe APA v7 en PDF:** Puede ser grupal (P.I.A) o individual. Si es grupal, el informe APA debe incluir detallado el nombre de cada integrante y qué tarea exacta hizo cada uno. Archivo guardado en su carpeta local como Docs/EEST1_LPR2026_ACT05_[ApellidoNombre_Grupo]_Informe_v1.0.0.pdf
- **Compilación (Local vs. Online):** El uso de VS Code y MSYS2 en la PC es exclusivo para quienes tengan notebook propia con permisos de administrador. (por una limitación de accesos en algunos equipos de laboratorio). Si las PCs de la escuela les bloquea o no les dejan compilar, es obligatorio usar OnlineGDB para correr el código (hagan las capturas correspondientes).
- **El repositorio:** Sin importar si compilaste en tu computadora o en OnlineGDB, el repositorio propio de GitHub con toda la estructura de carpetas (src/, docs/, etc.) y el código subido con su modificación debe estar creado obligatoriamente. No se aceptan carpetas sueltas.
- **Carpeta de Campo analógica y digital del P.I.A. y Planilla de Proyectos:** Registro obligatorio de todas las actividades relacionadas exclusivamente con el P.I.A., no se deben dejar campos vacíos, se deben justificar con textos coherentes apuntando al P.I.A. (los campos son: **Objetivos para la semana; Entregas Parciales (APA + Código + Capturas); Obstáculos Encontrados y Acciones de Próxima Semana**).

🔗 Enlaces Técnicos de Referencia (Links de Consulta):

- 🔗 Estructuras en C++ (cplusplus.com) - Guía rápida y oficial.
- 🔗 Punteros y asignación de memoria - Ejemplos interactivos.
- 🔗 Git Book en Español - Para resolver dudas de commits y ramas.

📅 Fecha de Entrega / Prórroga: 25 de Septiembre de 2026.